Practical Exam — SQL & MongoDB Queries

All 12 Problem Statements with Complete Queries

Problem Statement 01 — SQL DDL Statements

Database: Employee, Works, Company, Manages

Q1. Create all four tables with appropriate data types and constraints

```
-- Employee Table
CREATE TABLE Employee (
    employee_name VARCHAR2(50) PRIMARY KEY,
    street        VARCHAR2(100) NOT NULL,
    city          VARCHAR2(50)  NOT NULL
);

-- Works Table
CREATE TABLE Works (
    employee_name VARCHAR2(50),
    company_name  VARCHAR2(50),
    salary        NUMBER(10,2) NOT NULL,
    PRIMARY KEY (employee_name, company_name),
    FOREIGN KEY (employee_name) REFERENCES Employee(employee_name)
);

-- Company Table
CREATE TABLE Company (
    company_name VARCHAR2(50) PRIMARY KEY,
    city         VARCHAR2(50) NOT NULL
);

-- Manages Table
CREATE TABLE Manages (
    employee_name VARCHAR2(50),
    manager_name  VARCHAR2(50),
    PRIMARY KEY (employee_name),
    FOREIGN KEY (employee_name) REFERENCES Employee(employee_name)
);
```

Q2. Create a new table Department using Employee table (existing)

```
CREATE TABLE Department AS
SELECT employee_name, city
FROM Employee;
```

Q3. Create Emp_Backup with same structure as Employee but without data

```
CREATE TABLE Emp_Backup AS
SELECT * FROM Employee WHERE 1 = 0;
```

Q4. Add phone_number column to Employee table, then verify

```
-- Add column
ALTER TABLE Employee ADD phone_number VARCHAR2(15);

-- Verify
```

```
DESC Employee;
```

Q5. Increase the size of city column in Company table

```
ALTER TABLE Company MODIFY city VARCHAR2(100);
```

Q6. Drop phone_number column from Employee table

```
ALTER TABLE Employee DROP COLUMN phone_number;
```

Q7. Rename column salary to monthly_salary in Works table

```
ALTER TABLE Works RENAME COLUMN salary TO monthly_salary;
```

Q8. Insert rows in Company, then remove all rows without deleting structure

```
-- Insert sample rows
INSERT INTO Company VALUES ('First Bank Corporation', 'New York');
INSERT INTO Company VALUES ('Small Bank Corporation', 'Chicago');
INSERT INTO Company VALUES ('Big Bank Corporation', 'Mumbai');

-- Remove all rows (structure remains)
TRUNCATE TABLE Company;
```

Q9. Change salary type to DECIMAL(10,2) in Works table

```
ALTER TABLE Works MODIFY monthly_salary DECIMAL(10,2);
```

Note: In Oracle, DECIMAL(10,2) is treated as NUMBER(10,2).

Problem Statement 02 — SQL DML Statements (Insert, Select, Update, Delete)

Schema Setup — Department & Employee Tables

```
-- Create Department table
CREATE TABLE Department (
    Deptno   NUMBER PRIMARY KEY,
    Dname    VARCHAR2(20) NOT NULL,
    Location VARCHAR2(30)
);

INSERT INTO Department VALUES (10, 'Accounting', 'Mumbai');
INSERT INTO Department VALUES (20, 'Research',   'Pune');
INSERT INTO Department VALUES (30, 'Sales',       'Nashik');
INSERT INTO Department VALUES (40, 'Operations',  'Nagpur');

-- Create Employee table
CREATE TABLE Emp (
    Empno       NUMBER PRIMARY KEY,
    Ename       VARCHAR2(30) NOT NULL,
    Job         VARCHAR2(20),
    Mgr         NUMBER,
    Joined_date DATE,
    Salary      NUMBER(10,2),
    Commission  NUMBER(10,2),
    Deptno      NUMBER REFERENCES Department(Deptno),
    Address     VARCHAR2(50)
);

INSERT INTO Emp VALUES (1001,'Nilesh Joshi',   'Clerk',    1005, TO_DATE('17-DEC-
95','DD-MON-YY'), 2800,  600,  20,'Nashik');
INSERT INTO Emp VALUES (1002,'Avinash Pawar',  'Salesman', 1003, TO_DATE('20-FEB-
96','DD-MON-YY'), 5000,  1200, 30,'Nagpur');
INSERT INTO Emp VALUES (1003,'Amit Kumar',     'Manager',  1004, TO_DATE('02-APR-
86','DD-MON-YY'), 2000,  NULL, 30,'Pune');
INSERT INTO Emp VALUES (1004,'Nitin Kulkarni', 'President',NULL, TO_DATE('19-APR-
86','DD-MON-YY'), 50000, NULL, 10,'Mumbai');
INSERT INTO Emp VALUES (1005,'Niraj Sharma',   'Analyst',  1003, TO_DATE('03-DEC-
98','DD-MON-YY'), 12000, NULL, 20,'Satara');
INSERT INTO Emp VALUES (1006,'Pushkar Deshpande','Salesman',1003,TO_DATE('01-SEP-
96','DD-MON-YY'),6500,  1500, 30,'Pune');
INSERT INTO Emp VALUES (1007,'Sumit Patil',    'Manager',  1004, TO_DATE('01-MAY-
91','DD-MON-YY'), 25000, NULL, 20,'Mumbai');
INSERT INTO Emp VALUES (1008,'Ravi Sawant',    'Analyst',  1007, TO_DATE('17-NOV-
95','DD-MON-YY'), 10000, NULL, NULL,'Amaravati');
COMMIT;
```

Q1. Display employee information with explicit column names

```
SELECT Empno    AS "Employee Number",
       Ename    AS "Employee Name",
       Job      AS "Job Title",
       Mgr      AS "Manager ID",
       Joined_date AS "Date of Joining",
       Salary   AS "Salary",
       Commission AS "Commission",
       Deptno   AS "Department Number",
       Address  AS "Address"
FROM Emp;
```

Q2. Display unique jobs from the table

```
SELECT DISTINCT Job FROM Emp;
```

Q3. Change location of dept 40 to Banglore

```
UPDATE Department
SET    Location = 'Banglore'
WHERE  Deptno   = 40;
COMMIT;
```

Q4. Change the name of employee 1003 to Nikhil Gosavi

```
UPDATE Emp
SET    Ename = 'Nikhil Gosavi'
WHERE  Empno = 1003;
COMMIT;
```

Q5. Delete Pushkar Deshpande from employee table

```
DELETE FROM Emp
WHERE  Ename = 'Pushkar Deshpande';
COMMIT;
```

Q6. Display employees whose job is 'Manager' or 'Analyst' (using OR & IN)

```
-- Using OR
SELECT * FROM Emp
WHERE Job = 'Manager' OR Job = 'Analyst';

-- Using IN
SELECT * FROM Emp
WHERE Job IN ('Manager', 'Analyst');
```

Q7. Display employee name & dept number for dept 10, 20, 30, 40

```
SELECT Ename, Deptno
FROM   Emp
WHERE  Deptno IN (10, 20, 30, 40);
```

Q8. Find names and joined date of employees whose name starts with 'A'

```
SELECT Ename, Joined_date
FROM   Emp
WHERE  Ename LIKE 'A%';
```

Q9. Find employees having 'i' as second letter in their name

```
SELECT Ename
FROM   Emp
WHERE  Ename LIKE '_i%';
```

Q10. Find dept number and maximum salary where max salary > 5000

```
SELECT   Deptno, MAX(Salary) AS "Max Salary"
FROM     Emp
GROUP BY Deptno
HAVING   MAX(Salary) > 5000;
```

Problem Statement 03 — SQL DML Using JOINs

Schema: Employee, Works, Company, Manages (same as PS-01 with data inserted)

Sample Data for Joins

```
INSERT INTO Employee VALUES ('Alice',   '123 Main St', 'New York');
INSERT INTO Employee VALUES ('Bob',     '456 Oak Ave', 'Chicago');
INSERT INTO Employee VALUES ('Carol',   '789 Pine Rd', 'Gotham');
INSERT INTO Employee VALUES ('Dave',    '321 Elm St',  'New York');

INSERT INTO Company  VALUES ('First Bank Corporation', 'New York');
INSERT INTO Company  VALUES ('Small Bank Corporation', 'Chicago');

INSERT INTO Works VALUES ('Alice', 'First Bank Corporation', 12000);
INSERT INTO Works VALUES ('Bob',   'Small Bank Corporation', 8000);
INSERT INTO Works VALUES ('Carol', 'First Bank Corporation', 11000);
INSERT INTO Works VALUES ('Dave',  'First Bank Corporation', 9500);

INSERT INTO Manages VALUES ('Alice', 'Carol');
INSERT INTO Manages VALUES ('Bob',   'Alice');
COMMIT;
```

Q1. Find names of employees who work for First Bank Corporation

```
SELECT e.employee_name
FROM   Employee e
JOIN   Works    w ON e.employee_name = w.employee_name
WHERE  w.company_name = 'First Bank Corporation';
```

Q2. Find names and cities of employees who work for First Bank Corporation

```
SELECT e.employee_name, e.city
FROM   Employee e
JOIN   Works    w ON e.employee_name = w.employee_name
WHERE  w.company_name = 'First Bank Corporation';
```

Q3. Find names, street, and city of employees at First Bank Corporation earning > $10000

```
SELECT e.employee_name, e.street, e.city
FROM   Employee e
JOIN   Works    w ON e.employee_name = w.employee_name
WHERE  w.company_name = 'First Bank Corporation'
  AND  w.salary > 10000;
```

Q4. Find employees who earn more than EACH employee of Small Bank Corporation

```
SELECT e.employee_name
FROM   Employee e
JOIN   Works    w ON e.employee_name = w.employee_name
WHERE  w.salary > ALL (
    SELECT salary
    FROM   Works
    WHERE  company_name = 'Small Bank Corporation'
);
```

Q5. Find employees who earn more than the average salary of their company

```
SELECT w.employee_name, w.salary, w.company_name
FROM   Works w
JOIN  (SELECT company_name, AVG(salary) AS avg_sal
       FROM   Works
       GROUP BY company_name) avg_w
ON w.company_name = avg_w.company_name
WHERE w.salary > avg_w.avg_sal;
```

Q6. Find the company with the smallest payroll (total salary)

```
SELECT   company_name, SUM(salary) AS total_payroll
FROM     Works
GROUP BY company_name
HAVING   SUM(salary) = (
    SELECT MIN(SUM(salary))
    FROM   Works
    GROUP BY company_name
);
```

Q7. Find companies whose average salary is higher than First Bank Corporation's average

```
SELECT   w.company_name, AVG(w.salary) AS avg_salary
FROM     Works w
GROUP BY w.company_name
HAVING   AVG(w.salary) > (
    SELECT AVG(salary)
    FROM   Works
    WHERE  company_name = 'First Bank Corporation'
);
```

Q8. Give all employees of First Bank Corporation a 10% raise

```
UPDATE Works
SET    salary = salary * 1.10
WHERE  company_name = 'First Bank Corporation';
COMMIT;
```

Q9. Insert names and salaries of employees earning above average into HighEarners

```
CREATE TABLE HighEarners (
    employee_name VARCHAR2(50),
    salary        NUMBER(10,2)
);

INSERT INTO HighEarners (employee_name, salary)
SELECT employee_name, salary
FROM   Works
WHERE  salary > (SELECT AVG(salary) FROM Works);
COMMIT;
```

Q10. Delete employees who work for a company located in Gotham

```
DELETE FROM Employee
WHERE employee_name IN (
    SELECT w.employee_name
```

```
    FROM   Works w
    JOIN   Company c ON w.company_name = c.company_name
    WHERE  c.city = 'Gotham'
);
COMMIT;
```

Problem Statement 04 — SQL DML Using Subqueries

Same schema and data as Problem Statement 03.

Q1. Find names of employees who work for First Bank Corporation (Subquery)

```
SELECT employee_name
FROM   Employee
WHERE  employee_name IN (
    SELECT employee_name
    FROM   Works
    WHERE  company_name = 'First Bank Corporation'
);
```

Q2. Find names and cities of employees at First Bank Corporation (Subquery)

```
SELECT employee_name, city
FROM   Employee
WHERE  employee_name IN (
    SELECT employee_name
    FROM   Works
    WHERE  company_name = 'First Bank Corporation'
);
```

Q3. Names, street, city of First Bank Corp employees earning > $10000 (Subquery)

```
SELECT employee_name, street, city
FROM   Employee
WHERE  employee_name IN (
    SELECT employee_name
    FROM   Works
    WHERE  company_name = 'First Bank Corporation'
      AND  salary > 10000
);
```

Q4. Employees earning more than each Small Bank Corporation employee

```
SELECT employee_name
FROM   Works
WHERE  salary > ALL (
    SELECT salary
    FROM   Works
    WHERE  company_name = 'Small Bank Corporation'
);
```

Q5. Employees earning more than their company's average (Correlated Subquery)

```
SELECT employee_name, salary, company_name
FROM   Works w_outer
WHERE  salary > (
    SELECT AVG(salary)
    FROM   Works w_inner
    WHERE  w_inner.company_name = w_outer.company_name
);
```

Q6. Company with smallest payroll (Subquery)

```
SELECT company_name
FROM   Works
GROUP BY company_name
HAVING SUM(salary) = (
    SELECT MIN(total)
    FROM  (SELECT company_name, SUM(salary) AS total
           FROM   Works
           GROUP BY company_name)
);
```

Q7. Companies with average salary higher than First Bank Corporation's (Subquery)

```
SELECT company_name
FROM   Works
GROUP BY company_name
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM   Works
    WHERE  company_name = 'First Bank Corporation'
);
```

Q8. Give all First Bank Corporation employees a 10% raise (Subquery)

```
UPDATE Works
SET    salary = salary * 1.10
WHERE  employee_name IN (
    SELECT employee_name
    FROM   Works
    WHERE  company_name = 'First Bank Corporation'
);
COMMIT;
```

Q9. Insert above-average earners into HighEarners (Subquery)

```
INSERT INTO HighEarners (employee_name, salary)
SELECT employee_name, salary
FROM   Works
WHERE  salary > (SELECT AVG(salary) FROM Works);
COMMIT;
```

Q10. Delete employees working for companies located in Gotham (Subquery)

```
DELETE FROM Employee
WHERE employee_name IN (
    SELECT employee_name
    FROM   Works
    WHERE  company_name IN (
        SELECT company_name
        FROM   Company
        WHERE  city = 'Gotham'
    )
);
COMMIT;
```

Problem Statement 05 — Unnamed PL/SQL Block (Control Structures)

Schema Setup

```
CREATE TABLE Borrower (
    Rollin      NUMBER PRIMARY KEY,
    Name        VARCHAR2(50) NOT NULL,
    DateofIssue DATE NOT NULL,
    NameofBook  VARCHAR2(100) NOT NULL,
    Status      CHAR(1) DEFAULT 'I' CHECK (Status IN ('I','R'))
);

CREATE TABLE Fine (
    Roll_no NUMBER,
    Date    DATE,
    Amt     NUMBER(10,2),
    FOREIGN KEY (Roll_no) REFERENCES Borrower(Rollin)
);

-- Sample data
INSERT INTO Borrower VALUES (101, 'Rahul', TO_DATE('01-MAR-2025','DD-MON-YYYY'),
'DBMS', 'I');
INSERT INTO Borrower VALUES (102, 'Priya', TO_DATE('10-FEB-2025','DD-MON-YYYY'),
'OS',   'I');
COMMIT;
```

PL/SQL Block — Library Fine Calculation

```
DECLARE
    v_roll      Borrower.Rollin%TYPE;
    v_book      Borrower.NameofBook%TYPE;
    v_issue_dt  Borrower.DateofIssue%TYPE;
    v_days      NUMBER;
    v_fine      NUMBER := 0;
    v_status    Borrower.Status%TYPE;
BEGIN
    -- Accept input from user (replace values as needed)
    v_roll := &roll_no;
    v_book := '&book_name';

    -- Fetch issue date and status
    SELECT DateofIssue, Status
    INTO   v_issue_dt, v_status
    FROM   Borrower
    WHERE  Rollin     = v_roll
      AND  NameofBook = v_book;

    -- Calculate days elapsed
    v_days := TRUNC(SYSDATE) - TRUNC(v_issue_dt);
    DBMS_OUTPUT.PUT_LINE('Days elapsed: ' || v_days);

    -- Calculate fine
    IF v_days BETWEEN 15 AND 30 THEN
        v_fine := v_days * 5;
    ELSIF v_days > 30 THEN
        v_fine := (30 * 5) + ((v_days - 30) * 50);
    ELSE
        v_fine := 0;
    END IF;

    -- Update status to Returned
    UPDATE Borrower
```

```
    SET    Status = 'R'
    WHERE  Rollin = v_roll AND NameofBook = v_book;

    -- Insert fine record if applicable
    IF v_fine > 0 THEN
        INSERT INTO Fine VALUES (v_roll, SYSDATE, v_fine);
        DBMS_OUTPUT.PUT_LINE('Fine Amount: Rs. ' || v_fine);
    ELSE
        DBMS_OUTPUT.PUT_LINE('No fine applicable.');
    END IF;

    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Record not found for given roll no and book.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Problem Statement 06 — Cursors (proc_Grade using Explicit Cursor)

Schema Setup

```
CREATE TABLE Stud_Marks (
    Roll        NUMBER PRIMARY KEY,
    Name        VARCHAR2(50) NOT NULL,
    Total_Marks NUMBER NOT NULL
);

CREATE TABLE Result (
    Roll  NUMBER,
    Name  VARCHAR2(50),
    Class VARCHAR2(30)
);

INSERT INTO Stud_Marks VALUES (1, 'Aarav',   1200);
INSERT INTO Stud_Marks VALUES (2, 'Bhavna',  950);
INSERT INTO Stud_Marks VALUES (3, 'Chinmay', 860);
INSERT INTO Stud_Marks VALUES (4, 'Deepa',   780);
COMMIT;
```

Stored Procedure: proc_Grade using Explicit Cursor

```
CREATE OR REPLACE PROCEDURE proc_Grade IS
    -- Explicit Cursor
    CURSOR c_students IS
        SELECT Roll, Name, Total_Marks
        FROM   Stud_Marks;

    v_roll   Stud_Marks.Roll%TYPE;
    v_name   Stud_Marks.Name%TYPE;
    v_marks  Stud_Marks.Total_Marks%TYPE;
    v_class  Result.Class%TYPE;
BEGIN
    OPEN c_students;
    LOOP
        FETCH c_students INTO v_roll, v_name, v_marks;
        EXIT WHEN c_students%NOTFOUND;

        -- Categorize
        IF v_marks >= 990 AND v_marks <= 1500 THEN
            v_class := 'Distinction';
        ELSIF v_marks >= 900 AND v_marks <= 989 THEN
            v_class := 'First Class';
        ELSIF v_marks >= 825 AND v_marks <= 899 THEN
            v_class := 'Higher Second Class';
        ELSE
            v_class := 'Pass / Below Criteria';
        END IF;

        INSERT INTO Result VALUES (v_roll, v_name, v_class);
    END LOOP;
    CLOSE c_students;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Grading complete.');
END proc_Grade;
/

-- Execute the procedure
BEGIN
    proc_Grade;
```

```
END;
/

-- Verify results
SELECT * FROM Result;
```

Problem Statement 07 — Stored Procedure: proc_Grade

Same schema as PS-06. A standalone procedure (without cursor loop).

Stored Procedure (Simplified — processes one student at a time)

```
CREATE OR REPLACE PROCEDURE proc_Grade (
    p_roll  IN  Stud_Marks.Roll%TYPE,
    p_class OUT Result.Class%TYPE
) IS
    v_marks Stud_Marks.Total_Marks%TYPE;
    v_name  Stud_Marks.Name%TYPE;
BEGIN
    SELECT Name, Total_Marks
    INTO   v_name, v_marks
    FROM   Stud_Marks
    WHERE  Roll = p_roll;

    IF v_marks >= 990 AND v_marks <= 1500 THEN
        p_class := 'Distinction';
    ELSIF v_marks >= 900 AND v_marks <= 989 THEN
        p_class := 'First Class';
    ELSIF v_marks >= 825 AND v_marks <= 899 THEN
        p_class := 'Higher Second Class';
    ELSE
        p_class := 'Pass / Below Criteria';
    END IF;

    INSERT INTO Result VALUES (p_roll, v_name, p_class);
    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found.');
END proc_Grade;
/

-- Anonymous block to call procedure for all students
DECLARE
    v_class Result.Class%TYPE;
BEGIN
    FOR rec IN (SELECT Roll FROM Stud_Marks) LOOP
        proc_Grade(rec.Roll, v_class);
        DBMS_OUTPUT.PUT_LINE('Roll: ' || rec.Roll || ' => ' || v_class);
    END LOOP;
END;
/
```

Problem Statement 08 — Database Triggers

Schema Setup

```
CREATE TABLE Library (
    Book_id    NUMBER PRIMARY KEY,
    Book_name  VARCHAR2(100) NOT NULL,
    Author     VARCHAR2(50),
    Issue_date DATE,
    Status     VARCHAR2(20)
);

CREATE TABLE Library_Audit (
    Audit_id   NUMBER GENERATED ALWAYS AS IDENTITY,
    Book_id    NUMBER,
    Book_name  VARCHAR2(100),
    Author     VARCHAR2(50),
    Issue_date DATE,
    Status     VARCHAR2(20),
    Action     VARCHAR2(10),
    Changed_on DATE DEFAULT SYSDATE
);

INSERT INTO Library VALUES (1, 'DBMS Concepts', 'Korth',   SYSDATE, 'Available');
INSERT INTO Library VALUES (2, 'OS Principles', 'Galvin',  SYSDATE, 'Issued');
COMMIT;
```

Trigger 1: BEFORE UPDATE — log old values before update

```
CREATE OR REPLACE TRIGGER trg_library_before_update
BEFORE UPDATE ON Library
FOR EACH ROW
BEGIN
    INSERT INTO Library_Audit
        (Book_id, Book_name, Author, Issue_date, Status, Action, Changed_on)
    VALUES
        (:OLD.Book_id, :OLD.Book_name, :OLD.Author,
         :OLD.Issue_date, :OLD.Status, 'UPDATE', SYSDATE);
END;
/
```

Trigger 2: BEFORE DELETE — log old values before delete

```
CREATE OR REPLACE TRIGGER trg_library_before_delete
BEFORE DELETE ON Library
FOR EACH ROW
BEGIN
    INSERT INTO Library_Audit
        (Book_id, Book_name, Author, Issue_date, Status, Action, Changed_on)
    VALUES
        (:OLD.Book_id, :OLD.Book_name, :OLD.Author,
         :OLD.Issue_date, :OLD.Status, 'DELETE', SYSDATE);
END;
/
```

Trigger 3: AFTER INSERT — log new records after insert

```
CREATE OR REPLACE TRIGGER trg_library_after_insert
AFTER INSERT ON Library
```

```
FOR EACH ROW
BEGIN
    INSERT INTO Library_Audit
        (Book_id, Book_name, Author, Issue_date, Status, Action, Changed_on)
    VALUES
        (:NEW.Book_id, :NEW.Book_name, :NEW.Author,
         :NEW.Issue_date, :NEW.Status, 'INSERT', SYSDATE);
END;
/
```

Test the Triggers

```
-- Test UPDATE trigger
UPDATE Library SET Status = 'Returned' WHERE Book_id = 2;

-- Test DELETE trigger
DELETE FROM Library WHERE Book_id = 1;

-- Test INSERT trigger
INSERT INTO Library VALUES (3, 'CN by Tanenbaum', 'Tanenbaum', SYSDATE,
'Available');

COMMIT;

-- Verify audit log
SELECT * FROM Library_Audit ORDER BY Changed_on DESC;
```

Problem Statement 09 — MongoDB CRUD Operations

1. Create database Employee

```
use Employee
```

2. Create collection emp1 using createCollection method

```
db.createCollection("emp1")
```

3. Insert 4-5 documents in emp1 collection

```
db.emp1.insertMany([
  { eno: 1, ename: "Nilesh",  address: "Nashik",   sal: 8000  },
  { eno: 2, ename: "Avinash", address: "Nagpur",   sal: 12000 },
  { eno: 3, ename: "Niraj",   address: "Pune",     sal: 20000 },
  { eno: 4, ename: "Sumit",   address: "Mumbai",   sal: 4000  },
  { eno: 5, ename: "Ravi",    address: "Amaravati",sal: 3000  }
])
```

4. Display all documents

```
db.emp1.find().pretty()
```

5. Insert document with custom object id

```
db.emp1.insertOne({
  _id: ObjectId("507f1f77bcf86cd799439011"),
  eno: 6, ename: "Kiran", address: "Solapur", sal: 9500
})
```

6. Display all employees having salary > 5000

```
db.emp1.find({ sal: { $gt: 5000 } }).pretty()
```

7. Display all employees having salary < 15000

```
db.emp1.find({ sal: { $lt: 15000 } }).pretty()
```

8. Display employees with salary > 10000 AND < 20000

```
db.emp1.find({ sal: { $gt: 10000, $lt: 20000 } }).pretty()
```

9. Update salary of all employees by 10%

```
db.emp1.updateMany(
  {},
  [{ $set: { sal: { $multiply: ["$sal", 1.10] } } }]
)
```

10. Delete employee having salary < 5000

```
db.emp1.deleteMany({ sal: { $lt: 5000 } })
```

11. Rename collection emp1 to employee1

```
db.emp1.renameCollection("employee1")
```

12. Display employees whose name starts with 'n' (case-insensitive)

```
db.employee1.find({ ename: { $regex: /^n/i } }).pretty()
```

13. Sort names in ascending order

```
db.employee1.find({}, { ename: 1, _id: 0 }).sort({ ename: 1 })
```

Problem Statement 10 — MongoDB Aggregation & Indexing

Setup: Create and populate collections

```
use Library

db.Books.insertMany([
  { book_id: 1, book_title: "DBMS",        author_name: "Korth",
publish_year: 2010 },
  { book_id: 2, book_title: "OS",          author_name: "Galvin",
publish_year: 2018 },
  { book_id: 3, book_title: "CN",          author_name: "Tanenbaum",
publish_year: 2016 },
  { book_id: 4, book_title: "Algorithms",  author_name: "Korth",
publish_year: 2020 },
  { book_id: 5, book_title: "Java Basics", author_name: "Gosling",
publish_year: 2012 }
])

db.Products.insertMany([
  { product_id: 1, name: "Laptop",   price: 55000 },
  { product_id: 2, name: "Mouse",    price: 500   },
  { product_id: 3, name: "Keyboard", price: 1200  },
  { product_id: 4, name: "Monitor",  price: 12000 }
])
```

Q1. Create index on price field in Products

```
db.Products.createIndex({ price: 1 })
```

Q2. Create compound index on author_name and publish_year in Books

```
db.Books.createIndex({ author_name: 1, publish_year: -1 })
```

Q3. View all indexes in Books and Products

```
db.Books.getIndexes()
db.Products.getIndexes()
```

Q4. Drop the price index from Products

```
db.Products.dropIndex({ price: 1 })
```

Q5. Total number of books published by each author

```
db.Books.aggregate([
  { $group: { _id: "$author_name", total_books: { $sum: 1 } } },
  { $sort:  { total_books: -1 } }
])
```

Q6. Average price of all products

```
db.Products.aggregate([
  { $group: { _id: null, avg_price: { $avg: "$price" } } }
```

```
])
```

Q7. Maximum and minimum price in Products

```
db.Products.aggregate([
  { $group: {
    _id: null,
    max_price: { $max: "$price" },
    min_price: { $min: "$price" }
  }}
])
```

Q8. Count books published after 2015

```
db.Books.aggregate([
  { $match: { publish_year: { $gt: 2015 } } },
  { $count: "books_after_2015" }
])
```

Q9. Display only book_title and author_name using $project

```
db.Books.aggregate([
  { $project: { _id: 0, book_title: 1, author_name: 1 } }
])
```

Q10. Sort Books by publish_year descending

```
db.Books.aggregate([
  { $sort: { publish_year: -1 } }
])
```

Problem Statement 11 — MongoDB Queries (Restaurants)

Setup

```
use Restaurants

db.createCollection("REST1")

db.REST1.insertMany([
  { restaurant_id: 3001, name: "Spice Garden",
    address: { building: "12", street: "MG Road", city: "Mumbai", zipcode:
"400001" },
    cuisine_type: "Indian", score: 85 },
  { restaurant_id: 3002, name: "Dragon Palace",
    address: { building: "34", street: "Anna Salai", city: "Chennai", zipcode:
"600002" },
    cuisine_type: "Chinese", score: 92 },
  { restaurant_id: 3003, name: "Pasta House",
    address: { building: "56", street: "FC Road", city: "Pune", zipcode: "411004"
},
    cuisine_type: "Italian", score: 78 },
  { restaurant_id: 3004, name: "Tandoor Express",
    address: { building: "78", street: "Linking Rd", city: "Mumbai", zipcode:
"400050" },
    cuisine_type: "Indian", score: 88 },
  { restaurant_id: 3005, name: "Burger Barn",
    address: { building: "90", street: "Park St", city: "Kolkata", zipcode:
"700016" },
    cuisine_type: "American", score: 70 }
])
```

Q1. Display all documents in REST1

```
db.REST1.find().pretty()
```

Q2. Display restaurant_id, name, city, zipcode — exclude _id

```
db.REST1.find(
  {},
  { _id: 0, restaurant_id: 1, name: 1,
    "address.city": 1, "address.zipcode": 1 }
)
```

Q3. Restaurants with score > 80 and < 100

```
db.REST1.find({
  score: { $gt: 80, $lt: 100 }
}, { _id: 0, name: 1, score: 1 })
```

Q4. Update score of restaurant_id 3001 to 90

```
db.REST1.updateOne(
  { restaurant_id: 3001 },
  { $set: { score: 90 } }
)
```

Q5. Update cuisine_type of all Indian restaurants to 'Indian_Zatka'

```
db.REST1.updateMany(
  { cuisine_type: "Indian" },
  { $set: { cuisine_type: "Indian_Zatka" } }
)
```

Q6. Delete restaurant with restaurant_id 3003

```
db.REST1.deleteOne({ restaurant_id: 3003 })
```

Q7. Delete all restaurants with score less than 85

```
db.REST1.deleteMany({ score: { $lt: 85 } })
```

Problem Statement 12 — MongoDB Queries (Students)

Schema Setup

```
use School

db.students.insertMany([
  { student_id: 1, name: "Alice",  age: 21, gender: "Female",
    marks: { math: 95, science: 88, english: 82 } },
  { student_id: 2, name: "Bob",    age: 19, gender: "Male",
    marks: { math: 65, science: 70, english: 85 } },
  { student_id: 3, name: "Carol",  age: 22, gender: "Female",
    marks: { math: 78, science: 92, english: 88 } },
  { student_id: 4, name: "Dave",   age: 20, gender: "Male",
    marks: { math: 91, science: 85, english: 79 } },
  { student_id: 5, name: "Eva",    age: 18, gender: "Female",
    marks: { math: 55, science: 60, english: 72 } }
])
```

Q1. Find all male students

```
db.students.find({ gender: "Male" }).pretty()
```

Q2. Find students who scored more than 90 in math

```
db.students.find({ "marks.math": { $gt: 90 } }, { name: 1, "marks.math": 1, _id: 0
})
```

Q3. Find students aged 20 and above

```
db.students.find({ age: { $gte: 20 } }, { name: 1, age: 1, _id: 0 })
```

Q4. Find students whose English marks are between 80 and 90

```
db.students.find(
  { "marks.english": { $gte: 80, $lte: 90 } },
  { name: 1, "marks.english": 1, _id: 0 }
)
```

Q5. Count the number of female students

```
db.students.countDocuments({ gender: "Female" })
```

Q6. Find the student with the highest science score

```
db.students.find(
  {},
  { name: 1, "marks.science": 1, _id: 0 }
).sort({ "marks.science": -1 }).limit(1)
```

Q7. Update Bob's math score to 75

```
db.students.updateOne(
  { name: "Bob" },
  { $set: { "marks.math": 75 } }
)
```

Q8. Delete all students who scored below 70 in math

```
db.students.deleteMany({ "marks.math": { $lt: 70 } })
```

Q9. Aggregation — average marks in science for all students

```
db.students.aggregate([
  { $group: {
    _id: null,
    avg_science: { $avg: "$marks.science" }
  }}
])
```

Q10. Count students with marks > 80 in ANY subject

```
db.students.countDocuments({
  $or: [
    { "marks.math":    { $gt: 80 } },
    { "marks.science": { $gt: 80 } },
    { "marks.english": { $gt: 80 } }
  ]
})
```